

Progetto A2: Comunicazione Real-Time tra Processori Asimmetrici con OpenAMP

Tutor: Francesco Boccola **Co-tutor:** Marcello Cinque **Durata:** 4 settimane **Tipo:** Progetto di analisi. Diversi deliverable sono cose che dovrete scoprire voi, non che vi vengono fornite. Le domande di analisi nella Sezione 5 sono il cuore del progetto: il vostro compito è rispondere con evidenze alla mano.

1. L'argomento

Sapete già come funziona OpenAMP in teoria (remoteproc, RPMsg, la resource table, libmetal) e avete visto una demo di remoteproc. Questo progetto riguarda una direzione più recente costruita su quelle stesse fondamenta.

I SoC moderni come lo Zynq UltraScale+ affiancano un cluster Linux (Cortex-A53) a un cluster real-time (Cortex-R5F) che devono comunicare tra loro. La risposta classica di OpenAMP è RPMsg su shared memory. La direzione che studierete, l'**hypervisorless virtio**, permette invece ai due core di usare l'intero ecosistema di **dispositivi virtio** (rete, entropia, console, e così via) direttamente tra core, senza alcun hypervisor in mezzo. Un **Physical Machine Monitor (PMM)** in user space sul lato Linux fornisce i back-end virtio; l'RTOS (Zephyr) sull'R5 esegue i front-end virtio; comunicano attraverso una regione di shared memory, e un'interruzione inter-processore (IPI) sostituisce quello che normalmente sarebbe un VM-exit dell'hypervisor. Poiché i descrittori delle virtqueue vengono gestiti in modo trasparente, i driver virtio standard funzionano sostanzialmente senza modifiche.

Questa linea di lavoro è stata poi generalizzata in un protocollo di trasporto chiamato **virtio-msg**, con una variante **virtio-msg-amp** per la comunicazione tra core principale e coprocessore, portata avanti dal progetto **HVAC** di Linaro (Heterogeneous Virtio for Automotive Computing). Parte del vostro compito è collocare la demo che studiate dentro questa traiettoria: dove si trova oggi e dove sta andando la tecnologia.

Da notare: non si tratta semplicemente di "virtio reso real-time". Il punto è riutilizzare il modello di dispositivi virtio, standardizzato e maturo, tra core eterogenei senza un hypervisor. Una rilevanza per il real-time esiste (un core dedicato, nessun jitter dovuto allo scheduling dell'hypervisor) ma ha anche dei costi (latenza di notifica, copie dei buffer). Parte della vostra analisi è valutarlo onestamente.

2. Materiale di lettura suggerito (da fare nella prima settimana)

Due articoli per inquadramento e metodologia (entrambi consigliati):

- *Evaluating Latency in Multiprocessing Embedded Systems for the Smart Grid*, Energies (MDPI) 14(11):3322, 2021. <https://www.mdpi.com/1996-1073/14/11/3322> (<https://www.mdpi.com/1996-1073/14/11/3322>) (open access). Stessa famiglia hardware (A53 + R5), OpenAMP e Xen, con una metodologia chiara per la latenza real-time basata su cyclicttest.
- *Evaluating the OpenAMP framework in real-time embedded SoC platforms*, IEEE, 2021. <https://ieeexplore.ieee.org/document/9666157/> (<https://ieeexplore.ieee.org/document/9666157/>) Focalizzato sulle latenze introdotte da OpenAMP stesso (tempo tra impostazione e gestione di un'interruzione).

L'argomento vero e proprio (non esiste ancora un articolo accademico, quindi questi materiali sono essenziali):

- Blog OpenAMP sull'hypervisorless virtio: https://www.openampproject.org/docs/blogs/HypervisorlessVirtioBlog_Feb2021.pdf (https://www.openampproject.org/docs/blogs/HypervisorlessVirtioBlog_Feb2021.pdf)
- Presentazione allo Zephyr Dev Summit 2022: <https://www.openampproject.org/news/hypervisorless-virtio-zephyr-dev-summit> (<https://www.openampproject.org/news/hypervisorless-virtio-zephyr-dev-summit>)
- Quick reference sul meccanismo (da leggere con attenzione: IPI al posto del VM-exit, bounce buffer): https://openamp.readthedocs.io/en/latest/hypervisorless_virtio_zcu102/README_hypervisorless_virtio.html (https://openamp.readthedocs.io/en/latest/hypervisorless_virtio_zcu102/README_hypervisorless_virtio.html)
- Dove sta andando (virtio-msg / HVAC): <https://linaro.atlassian.net/wiki/spaces/HVAC/overview> (<https://linaro.atlassian.net/wiki/spaces/HVAC/overview>)

Materiale di riferimento:

- Documentazione OpenAMP: <https://openamp.readthedocs.io/en/latest/index.html> (<https://openamp.readthedocs.io/en/latest/index.html>)
- Dettagli del protocollo RPPMsg: https://openamp.readthedocs.io/en/latest/protocol_details/rppmsg.html (https://openamp.readthedocs.io/en/latest/protocol_details/rppmsg.html)
- Video tutorial: https://www.youtube.com/watch?v=R_5DOlvo7tI (https://www.youtube.com/watch?v=R_5DOlvo7tI) e <https://www.youtube.com/watch?v=kjoQ2Gidbuo> (<https://www.youtube.com/watch?v=kjoQ2Gidbuo>)
- Progetto Zephyr: <https://www.zephyrproject.org/> (<https://www.zephyrproject.org/>)

3. Il modo più rapido per avere la demo funzionante (senza board)

La demo gira interamente in QEMU tramite un container già pronto, che emula un intero ZCU102 (entrambi i cluster):

```
docker run -it openamp/demo-lite
gemu-zcu102 ./demo4
```

Lasciate fare l'autoboot a U-Boot. Sull'A53 ottenete una shell PetaLinux; sull'R5 una shell Zephyr su una seconda UART.

Nella shell Zephyr, `device list` mostra `virtio0/virtio1/virt-rng/virt-net`, e `net ping -c 10`

`192.168.200.254` fa il ping del back-end Linux. Questa è la demo, interamente osservabile su un portatile. Istruzioni:

https://openamp.readthedocs.io/en/latest/hypervisorless_virtio_zcu102/README_demo.html

(https://openamp.readthedocs.io/en/latest/hypervisorless_virtio_zcu102/README_demo.html)

Nota sull'hardware: per il progetto di base non serve alcuna board. Una ZCU102 fisica è disponibile in laboratorio per gli obiettivi opzionali della Sezione 6.

4. Il piano delle quattro settimane

Come dividere il lavoro tra voi due è una vostra scelta. Una buona abitudine è convergere su un artefatto condiviso alla fine di ogni fase e affrontare in coppia le parti più difficili. Tenete un logbook condiviso fin dal primo giorno.

Settimana 1: Fondamenta e primo avvio

- Completate la lettura suggerita (Sezione 2) e guardate i video.
- Fate girare `openamp/demo-lite` in QEMU e riproducete l'output di entropia e il `net ping` bidirezionale.
- Capite le due metà del sistema: sul lato Linux/A53, come viene caricato il firmware dell'R5 (`remoteproc`), come parte il PMM, come viene configurata la rete tap e come i back-end virtio vengono esposti sulla shared memory (`virtio-mmio`); sul lato Zephyr/R5, come vengono individuati `virt-rng` e `virt-net` e come viene segnalato il PMM tramite IPI.

Deliverable D1 (mappa del sistema): un diagramma più una descrizione scritta della demo: i due core, il PMM e i suoi back-end, la regione di shared memory, i dispositivi virtio-mmio e le notifiche IPI. Ripercorrete a parole cosa succede dall'avvio fino a un ping riuscito.

Settimana 2: Analisi a livello di codice

- Compilate il sample Zephyr dai sorgenti (questa è la parte più ostica; mettete in conto del tempo per la toolchain).
Sample: `hvl_net_rng_reloc` nel repository `zephyr-openamp-staging`, branch `virtio-exp`:
https://github.com/OpenAMP/zephyr-openamp-staging/tree/virtio-exp/samples/virtio/hvl_net_rng_reloc
(https://github.com/OpenAMP/zephyr-openamp-staging/tree/virtio-exp/samples/virtio/hvl_net_rng_reloc).
- Individuate il driver virtio-mmio e la "colla" specifica dell'hypervisorless (i driver virtio di OpenAMP sul branch `virtio-exp`; per i percorsi esatti vedete la quick reference della Sezione 2).
- Tracciate una lettura di `virt-rng` e un pacchetto di `virt-net` dall'inizio alla fine: dove si trova la `virtqueue`, dove i descrittori vengono aggiunti e consumati, dove viene sollevata l'IPI, dove avviene la copia nel bounce buffer.

Deliverable D2 (analisi del codice): mappate l'architettura di D1 su file e funzioni reali. Distinguate chiaramente cosa è **virtio standard** da cosa è **specifico dell'hypervisorless** (le weak function dipendenti dall'OS, l'IPI al posto del VM-exit, i bounce buffer nella memoria precondivisa).

Settimana 3: Esperimenti e modifiche

- Fate modifiche controllate per dimostrare la vostra comprensione: aggiungete strumentazione/print su entrambi i lati; modificate un messaggio o il payload del ping; usate il dispositivo rng in modo diverso; se possibile, provate a osservare o abilitare un altro dispositivo virtio.
- Prendete misure di tempo approssimative sul percorso di IPC (per esempio il round-trip del ping, o il tempo tra notifica e gestione). **Avvertenza importante:** i tempi in QEMU non sono rappresentativi dell'hardware reale, quindi riportate comportamento funzionale e tendenze relative, non numeri real-time assoluti. Usate la metodologia degli articoli come guida su *come* misurare, esplicitando però il limite di QEMU.

Deliverable D3 (registro degli esperimenti): cosa avete cambiato, come ha reagito ciascun lato, ed eventuali osservazioni temporali con le dovute avvertenze.

Settimana 4: Sintesi

- Rispondete alle domande di analisi della Sezione 5 con le evidenze raccolte.
- Collocate la demo rispetto all'RPMsg classico e rispetto alla direzione virtio-msg / HVAC.

Deliverable D4 (relazione finale + breve presentazione): l'analisi scritta e un insieme di slide conciso.

5. Le domande di analisi (queste risposte SONO il progetto)

1. **Quale virtio viene effettivamente usato?** Quali dispositivi virtio e quali parti della specifica virtio vengono esercitati dalla demo, e quali sono stubbati o inutilizzati?
 2. **Cosa lo rende "hypervisorless"?** Esattamente quali assunzioni del virtio standard hanno dovuto essere adattate, e come (IPI vs VM-exit, PMM in user space, memoria precondivisa, bounce buffer, gestione dei descrittori)?
 3. **Cosa offre virtio rispetto a RPMsg qui?** Confrontate l'approccio hypervisorless virtio con il classico RPMsg-suvring: cosa diventa più semplice o riutilizzabile, e a quale costo?
 4. **Quali sono i limiti attuali?** Considerate sicurezza (il back-end può accedere alla memoria del remoto), prestazioni (overhead delle copie), layout di memoria statico, copertura dei dispositivi, e il fatto che si tratta di un branch sperimentale.
 5. **Qual è la rilevanza real-time, onestamente?** Dove eliminare l'hypervisor aiuta davvero, e dove questo approccio aggiunge latenza o jitter? Valutate criticamente l'affermazione "rimuove le limitazioni real-time".
 6. **Dove sta andando?** Riassumete l'evoluzione verso virtio-msg e il progetto HVAC, e cosa implica per i sistemi AMP industriali e automotive.
-

6. Obiettivi opzionali (solo se le settimane da 1 a 3 procedono bene)

- **Hardware reale (ZCU102):** fate girare la demo sulla ZCU102 fisica che abbiamo in laboratorio. La demo QEMU ha già come target la ZCU102, quindi gli artefatti di build ufficiali sono validi; il lavoro consiste nel portarla sulla board reale, confermare il comportamento e, soprattutto, prendere misure di tempo reali (che QEMU non può darvi). È il modo naturale per trasformare le avvertenze sui tempi della Settimana 3 in numeri concreti.
 - **Un confronto di latenza più netto:** RPMsg classico contro il percorso di rete dell'hypervisorless virtio, idealmente sulla ZCU102 per avere numeri assoluti.
-

7. Avvertenze pratiche

- Il codice vive sul branch sperimentale `virtio-exp`, quindi aspettatevi qualche difficoltà. Il container `demo-lite` già pronto funziona out of the box; compilare dai sorgenti (west, l'SDK di Zephyr, la build rilocabile) è dove si perde tempo.

- La rete tap richiede privilegi; è gestita all'interno del container, quindi conviene lavorare al suo interno invece di ricomporre i pezzi all'esterno.
- I repository di staging si spostano. Se un link vi dà errore 404, navigate a partire dall'organizzazione OpenAMP su GitHub (<https://github.com/OpenAMP> (<https://github.com/OpenAMP>)) e selezionate il branch `virtio-exp`.
- [IMPORTANTE] Tenete un repository git condiviso con un logbook fin dal primo giorno: versioni, comandi, osservazioni e dati grezzi. In un progetto di analisi, il logbook è estremamente importante.